

WebJump: AR-facilitated Distributed Display of Web Pages

Xin Zeng
Beijing Key Laboratory of Mobile
Computing and Pervasive Device
Institute of Computing Technology,
Chinese Academy of Sciences, UCAS
Beijing, China
zengxin18z@ict.ac.cn

Xiaoyu Wang
Yuanpei College, Peking University
Beijing, China
1800017830@pku.edu.cn

Zhengtai Gou
Pervasive Human Computer
Interaction Laboratory, Tsinghua
University
Beijing, China
jizt19@mails.tsinghua.edu.cn

Yiqiang Chen
Beijing Key Laboratory of Mobile
Computing and Pervasive Device
Institute of Computing Technology,
Chinese Academy of Sciences, UCAS
Beijing, China
yqchen@ict.ac.cn

Tengxiang Zhang
Beijing Key Laboratory of Mobile
Computing and Pervasive Device
Institute of Computing Technology,
Chinese Academy of Sciences, UCAS
Beijing, China
zhangtengxiang@ict.ac.cn

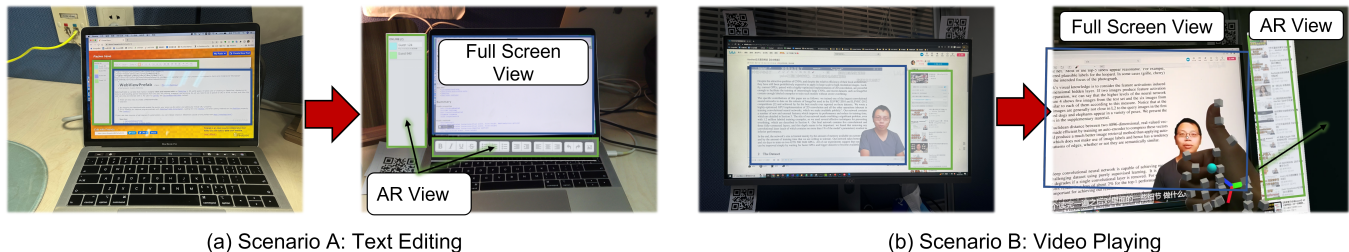


Figure 1: Two scenarios in which WebJump improves user experience. In the text editing scenario (a), the toolbar is displayed on the keyboard for faster access while the working area is in full-screen mode; in the video playing scenario (b), the user performs a thumb-up gesture to like the video, gazes at and air-clicks the next video to play.

ABSTRACT

Head-mounted displays (HMD) like AR glasses support powerful 3D displays and intuitive input modalities. However, there is a lack of collaboration between the HMD and other displays like PC monitors. In this paper, we propose WebJump, a software tool that analyzes HTML web pages and enables UI elements ‘jump’ from the PC monitor to the HMD. This way, contents like high-resolution images and long texts can take full advantage of the high-quality monitor display. Auxiliary contents like sidebars and menus, on the other hand, are offloaded to the HMD for easier access albeit with a lower display quality. We describe our software tool and explain in detail its implementation. We develop two applications to demonstrate WebJump’s feasibility and potential.

CCS CONCEPTS

• **Human-centered computing** → **User interface management systems**; **Gestural input**.

KEYWORDS

mixed reality, distributed display, web pages

ACM Reference Format:

Xin Zeng, Xiaoyu Wang, Zhengtai Gou, Yiqiang Chen, and Tengxiang Zhang. 2023. WebJump: AR-facilitated Distributed Display of Web Pages. In *Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems (CHI EA '23)*, April 23–28, 2023, Hamburg, Germany. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3544549.3585669>

1 INTRODUCTION AND BACKGROUND

Head-mounted displays (HMD) like augmented reality glasses are gaining traction. Such displays can render digital content like images, text, and control widgets while allowing users to see the physical world simultaneously. Instead of 2D displays like PC monitors and phone screens, HMDs can render 3D digital content following the user. The multi-modal input gesture, gaze, and speech make the interaction more efficient and convenient. Thus, many claim that HMD will be the final screen replacing all existing screens. However, that vision is still far ahead of us since HMD’s display quality

is usually inferior to existing screens. This will significantly lower the user experience, especially when displaying high-quality multimedia content like 4K images and videos. The field of view (FOV) of existing HMDs is also quite limited, greatly reducing working productivity (e.g., when multiple documents are cross-referenced frequently). It is then beneficial to combine the low-quality head-mounted display and the high-quality screens to unleash the power of both. Collaboration of the HMD and existing screens enables digital content 'jump' freely between displays, making it possible to optimize digital content distribution across screens. For example, a word processor's tools bar can be offloaded to the HMD and rendered near the keyboard. This way, the buttons for the tools are easier to access, and the working area is maximized on the PC monitor to improve editing efficiency. A video streaming website can play the 4K video full-screen on a PC monitor while the comment section is rendered by the HMD beside the monitor.

This is in fact a distributed user interface (DUI) between the PC and the HMD. Previous research studies distributed interfaces across multiple PC [1], mobile [5], smartwatch [12] and AR [6]. As HMDs are gaining traction, however, it becomes important to achieve real-time UI elements redistribution between HMD and conventional screens like PC monitors. Also, most previous research only works within customized software. Analyzing and extracting digital components of one or more commercial PC software is still difficult.

In this paper, we evaluate the concept of AR-facilitated cross-screen display on HTML web pages instead. Today's powerful web browsers allow users to search for information, watch the news, stream videos, and edit documents. They are even used by some operating systems (e.g., ChromeOS) as the principal user interface. We propose WebJump, a software tool that redistributes elements of a webpage between a PC monitor and a Hololens 2. On the PC side, WebJump analyzes an HTML-based web page displayed on its monitor using a crawler script and sends marked components to the HMD; on the HMD side, WebJump renders the received components and notifies the PC of any relevant interaction events. This way, content like images, videos, or text that demands high resolution and fast refresh rate can occupy the entire PC monitor. At the same time, auxiliary UI elements like menus and control widgets move to an HMD for easier viewing, access, and manipulation. The utility of the proposed system is illustrated in Figure 1.

We design a tagging scheme for web designers (or users) to customize the re-distribution strategy. Specifically, *what* to distribute, *where* to display, and *how* to interact. A Java script then analyzes these tags and sends the information to HMD. On the HMD side, a WebView browser renders the selected web elements at designated locations with the expected size. At running time, HMD monitors and sends interactive events (e.g., click on a tool button) back to the PC. And the PC also replicates the received event to complete an interaction.

The contributions of our work are:

- We propose redistributing the digital content between the PC monitor and the HMD for optimal screen usage and intuitive input.
- We designed and implemented WebJump, a software tool to analyze web pages and display web content on a PC monitor and an AR headset.

- We built two demos for frequent application scenarios to demonstrate the use cases and advantages of the proposed system.

2 RELATED WORK

Significant effort has been put into prototyping cross-device interaction and visualization platforms by the HCI community. In our related works, we follow the thread to cover topics of distributed user interface (DUI), and review how Augmented Reality plays its role in such cross-device scenarios.

2.1 Distributed User Interface

The need for another screen arises when the digital workload becomes overwhelming, and has since stimulated the research and developments of Distributed User Interface (DUI). As a pioneering attempt to interconnect different devices, EMME [2] created a collaborative augmented environment where users wear see-through HMDs to interact with virtual objects and move 3D contents across laptops. Later researches extend to more sorts of devices, including phones [16], smart watches [14] and televisions [11]. Schirra and Bentley [12] discussed the interaction between commercial smartphones and smartwatches and explored the smartwatch's role as an additional screen.

A key obstacle in the development of DUI is the establishment of a cohesive framework capable of accommodating diverse devices, inputs and outputs, while ensuring proper synchronization. Vistribute [4] implements a framework for distribution of UI elements across devices based on user preference and device, which minimizes user's design effort by semi-automatic page segmentation and distribution. Weave [3] proposes a scripting framework for distributing UI outputs across mobile and wearable interaction. SurfaceFleet [1] supports fine-grained cross-device resource sharing, although users still have to operate on different computers for cross-device operations. Ubii [5] includes physical objects within AR recognition and content rendering to build ubiquitous interface and interaction. However, it only supports file-level transmission between devices. Panelrama [15] proposes to categorize device characteristics and dynamically change UI allocation to best-fit devices.

While these tools have greatly facilitated developers, one sometimes neglected aspect is compatibility with existing content, especially web applications. Since most of the digital content that users encounter daily is from traditional platforms that are not designed for multi-interface usage, it thus becomes an important challenge to explore ways of DUI for web pages, which is a good cross-system, cross-interface and cross-device platform. XDBrowser [8] uses a proxy server to modify web pages with cross-device annotations that determine how elements should be distributed across devices and communicate with each other. The client renders the modified web page using a custom browser with gesture and voice command support. The gesture recognition module relies on OpenCV to detect user gestures, such as swiping, pointing, and tapping, and sends them to the server for interpretation. XDBrowser2 [7] follows a similar architecture but adds a semi-automatic generation component that uses HTML5 APIs to access web page structure and content. The component analyzes existing pages, identifies

cross-device elements (menus, forms, tables, charts, etc.), and creates annotations based on their properties and functions. Users can customize the generated interfaces with gestures and voice commands on their devices. Although XDBrowser's exploration of distributed interfaces based on HTML technology has inspired future research, it mainly focuses on traditional devices and does not take into account the properties and locations of elements in distributed interfaces.

2.2 Leveraging Augmented Reality in DUI scenario

AR HMDs serve as auxiliary input/output source or extended display in various cross-device scenarios. Multiple sensors including a gaze tracker and depth camera make it possible to detect interactive gestures and locate different objects. For example, HoloDoc [6] enables a mixed-reality workspace that harnesses physical and digital content. Using interactive paper with an Ncode pattern, the HoloLens device can recognize documents in addition to hand gestures and pen strokes. One distinguished advantage of AR HMDs in cross-device utility is the flexible display. AR HMDs can present 3D virtual objects in the entire physical spatial space or extend existing screens like PC monitors and smartphone screens. BISHARE [16] uses HoloLens to exhibit virtual models and extends 2D displays in peripheral space while interacting bilaterally. Normand and McGuffin [9] propose extended screen space for smartphones in augmented reality, as contents can be displayed outside the phone. Gluey [13] proposes a concept to copy resources into the head-mounted display and then paste them onto another device for cross-device resource manipulation. Gluey leverages HMD to unify interaction input and present interactive content, serving for workspace presentation and device control. For specific use cases, DesignAR [10] uses augmented reality for 3D modeling, where users can observe 3D objects floating above a tabletop screen and ready for manipulation.

In this work, we employ HMDs to build a distributed user interface in collaboration with traditional PC displays, fully exploiting their capability and advantage in the challenging scenario of DUI.

3 SYSTEM DESIGN AND IMPLEMENTATION

WebJump re-distributes web page elements across PC and HMD to make the best use of inputs and displays on both devices, and accordingly would require collaboration in-between. We designed a tagging scheme for HTML-based web pages to indicate how each web element should be displayed. When the user opens a web page on the PC, our tool analyzes the source code and looks for the WebJump tags. It then immediately initiates communication with the HMD, sending over elements tagged for the HMD. The HMD then renders the web page elements referring to the tag information. The HMD also monitors interaction events with such web elements. Once an event is triggered, it sends the interaction event back to the PC. Our tool then replicates the events on the PC. In this way, WebJump supports intuitive PC interaction with input modalities only available on the HMD. The whole architecture of WebJump is illustrated in Figure 2.

3.1 Display Configurations through Web Element Attributes

WebJump requires web designers (or users) to customize the display configurations of different parts of a web page, including *what* to distribute, *where* to display, and *how* to interact. We design identifiers and configurations as attributes of different elements of the web page's DOM tree for easier parsing. Users can customize system behavior by assigning tags to corresponding elements. More specifically, WebJump related element attributes are {"jump identifier": y , "anchor object": o , "relative position", (p_x, p_y, p_z) , "visualization description": a_{out} , "interaction action": a_{in} }. Attributes of the lower-level elements will supersede those of the high-level elements. We explain each of the attributes below and summarize them in Table 1.

Jump Identifier identifies elements that require distributed display. Essentially, it is a binary tag that decides whether this element needs to be targeted and processed. Our tool searches for the jump identifier for each element. Once found, it will further look for other WebJump related attributes.

Anchor Object is the object the element is attached to when displayed on the HMD. HMD offers more ways to display website elements. One way is to attach the element to a physical object. For example, a toolbar can be anchored to the keyboard, a button around the left hand, and a comment section beside the screen. The anchor self indicates that the element stays on the PC. In this paper, we use QR codes to locate anchors. It is also possible to identify objects using computer vision techniques, which we leave for future work.

Relative Position determines the exact rendering coordinates in HMD relative to the anchor. These are numerical values for fine-grained adjustment.

Visualization Description describes user-desired display or rendering effects. For example, on PC, we can *maximize* a streaming video to fill the entire screen; in HMD, we should *resize* the element properly for the best viewing experience.

Interaction Action specifies which kind of inputs the system takes and monitors, and defines how the HMD inputs are mapped to PC operations. Typically, HMD inputs include click, gaze, and gesture. Gaze indicates the PC's mouse cursor location, which an air-click translates to a mouse click event. WebJump also detects fine-grained hand shapes and motions and maps them to specific functions specified in this attribute. The current WebJump system supports *like* (thumb-up) and *dislike* (thumb-down). *None* is used if the element does not require inputs.

3.2 Implementation

Our prototype system uses HoloLens2 as the HMD and our software is developed using unity, Mixed Reality Toolkit¹ and 3D WebView². We built WebJump to work with Google Chrome. We use Python and its dependency packages including Selenium³, xpath, etree and

¹<https://github.com/microsoft/MixedRealityToolkit-Unity>

²<https://developer.vuplex.com/webview/overview>

³<https://www.selenium.dev/>

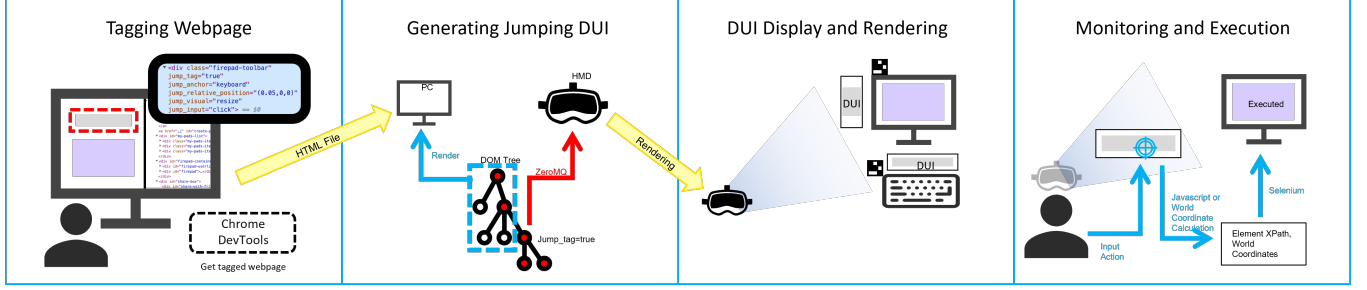


Figure 2: The overall framework of WebJump. (a): Web designers (or users) tag the webpage to customize the re-distribution strategy. (b): PC side generates jumping DUI based on the (previously cached) configuration and send them over to HMD. (c): HMD side renders DUI at specific position with specified size as instructed. (d) HMD side keeps monitoring interaction event and send them back to PC, PC side replicates the event accordingly.

Table 1: Supported attribution and their available values in WebJump.

Field	Notation	HTML attribute	Values
Jump Identifier	y	jump tag	true, false
Anchor Object	o	jump anchor	self, screen, left-hand, right-hand, ...
Relative position	(p_x, p_y, p_z)	jump relative position	$\mathbb{R}$
Visualization Description	a_{out}	jump visual	maximize, resize, none
Interaction Action	a_{in}	jump input	click, gaze, gesture, none

ZeroMQ⁴. We detail the implementation of WebJump in four parts: Webpage Tagging, DUI Generation, DUI Display, and Interaction Monitoring.

3.2.1 Webpage Tagging. The web designer needs to tag the web elements with WebJump attributes. It is even possible for users to tag an existing web page. In our current version, the user first opens up the HTML source code through developer tools of the browser, then adds expected values for each specific field on desired elements, and saves the modified code into a local directory. Users then launch the WebJump system with the same webpage URL, and the software will re-load and parse the corresponding schema (modified code). Note that the schema is website-specific but reusable. Users only need to configure once, and the software can automatically match and load them. The regular expression is used here to match multiple URLs that belong to the same website against a shared schema.

Users who lack coding knowledge may face limitations in their ability to perform tagging in a web-based system. This is because they may struggle to comprehend the underlying structure of the web page and select appropriate tags for specific elements. However, potential solutions to overcome this limitation will be discussed in the future work section. By developing more user-friendly interfaces and automatic tagging systems, we can make it easier for all users to tag UI elements and contribute to the development of web-based systems.

3.2.2 DUI Generation. We use Python to parse the tagged webpage code. Specifically, all jumping elements can be obtained simultaneously with the Selenium package by `document.querySelectorAll`

(`'[JumpTag="True"]'`). To get a clear view of jumping elements, the system generates codes of the updated pages with jumping elements, while removing all other irrelevant elements. This is accomplished by modifying the style attribute of jumping elements as: `position:fixed;left:0px;top:0px;background:rgb(255, 255, 255); and z-index:100000000;width:100vw;height:100vh; overflow-y:scroll;`. In our practical implementation, we found that certain floating elements in several specific websites can not be covered by the above rule. The users then need to manually set the jump tag of these irrelevant elements to false, and the system will hide them proactively. Such rules will also be cached locally for future reuse so the user only needs to modify the generated page once. Finally, the PC will send the URL, WebJump attributes, and the generated jumping elements to HMD for rendering.

3.2.3 DUI Display. The webpage in HMD should be resized proportionally to the size of jumping elements so that the elements will fill the entire page just right. HMD first loads the PC-generated webpage (with only jumping elements) with a large enough resolution. It then obtains the size of elements. Based on the size and the pre-defined output description a_{out} , we can calculate the required page height and resize the page accordingly. Finally, we re-adjust the resolution to scale the entire page so that jumping elements can fill it.

The HMD also needs to calculate the spatial position of the page. The system uses QR codes to indicate each anchor object and obtain their world coordinates. Our prototype uses MRTK to track QR codes.

An anchor object is a 3D box with center coordinates, vertex coordinates, and rotation relative to the headset. According to the configured schema, we can then add relative position (p_x, p_y, p_z)

⁴<https://zeromq.org/>

upon the anchor to get the final world coordinates and place the page. Each element is accordingly set as a sub-object inside the rendered page.

Currently, the distributed UI process is triggered automatically. However, this may not always provide the optimal user experience. Therefore, we intend to explore and determine the most efficient and user-friendly way to trigger UI elements in a distributed system through future work.

3.2.4 Interaction Monitoring. The HMD monitors interaction events of each rendered element according to the pre-configured schema, e.g., click, gesture or gaze. If an event is triggered, the world coordinates of the interacting element (denoted as p_{world}), are accessed by MRTK. However, to pass the information of the element to PC, we need to calculate the element's coordinates relative to the webpage (denoted as $p_{web} = (x, y)$). Assuming the normalized width vector $\vec{w}$ and height vector $\vec{h}$ of the rendered page, the transformation between p_{world} and $p_{web} = (x, y)$ can be derived as:

$$p_{world} = x * \vec{w} + y * \vec{h} \quad (1)$$

Thus we get the relative position of the element on the webpage in AR. By executing JavaScript in 3D WebView, we can obtain the XPath of the element, and it is then passed to the PC to replicate this user-input event using Selenium and JavaScript.

4 USAGE SCENARIO

We implemented two application scenarios to show how WebJump can improve the interaction experience when using a PC.

4.1 Text Editing

It is now common to edit documents online using a web browser. When editing a large document on a small screen (e.g., laptop), the user must scroll back and forth frequently. Also, document formatting requires switching between the mouse and the keyboard. WebJump offloads the display of elements like the user list and the toolbar to a Hololens 2 headset, which enables a full-screen display of the working area. This way, more text is displayed on the PC monitor, so fewer scrolls are needed. The formatting toolbar is placed near the keyboard (similar to a Mac touch bar⁵) to avoid hand-switching between the keyboard area and the mouse.

4.2 Video Playing

Online video streaming is very popular these days. Many online video streaming websites (e.g., YouTube) simultaneously display the video, the comments, and the recommended video list to meet users' versatile demands. For example, people tend to leave comments while they watch the video. They will also stay on the website for a longer time with accurate video recommendations. However, such sections take up a large portion of the screen, making the video small. With WebJump, the video can be played in full-screen mode while a Hololens 2 headset displays the comments and recommended videos. The user can use a thumb-up gesture to like the currently playing video, or gaze at the recommended video and click to play. In this way, WebJump makes the best use of the

high-quality PC monitor while taking full advantage of the AR headset's intuitive gesture and gaze inputs.

5 CONCLUSION AND FUTURE WORK

In this paper, we propose WebJump, an AR-facilitated system that redistributes elements of a web page between a PC monitor and an HMD. The system optimizes screen usage according to web designers' expectations while offering easy access and intuitive interaction.

Although WebJump makes a novel and significant attempt towards the distributed display of contents with HMD integration, our current implementation can still be further improved. We summarize the following aspects as promising future directions.

- **Tagging Interface.** The tagging schema to customize the re-distribution strategy currently requires manual examination of the webpage source code. It would greatly improve the user experience and lower the interaction burden to provide a more user-friendly interface. The interface should correctly abstract the structure of a webpage from its source code, and allow users to customize the re-distribution strategy directly through GUI instead of source code.
- **Automatic Tagging.** Another very promising direction would be automatic and adaptive tagging. Currently, WebJump covers frequent user scenarios, and the schema can be repeatedly imported. However, for many long-tail scenarios, users would also like to employ a natural, adaptive re-distribution experience using the system but without the cost of tagging every specific website. The capabilities demonstrated by powerful general AI models like ChatGPT⁶ also make this possible. Besides, the re-distribution strategy can be delicate to customize and adjust, which requires time and energy.
- **Comprehensive Representation.** WebJump in its current version makes the first attempt towards transferring UI elements from a web page environment to 3D space, creating an "extended screen" effect. However, there might be other alternative or more complete solutions to explore a more comprehensive representation of UI elements in AR space. This can include exploring the appearance, 3D volume, and interactive possibilities of UI elements. By doing so, developers can identify the benefits of separating elements in space and create a more immersive and engaging experience for users.
- **Interaction Experience.** Although WebJump has augmented the ways of interaction by integrating multi-modal inputs and convenient gestures in HMD, we believe the potential of powerful HMDs is still not fully revealed. Currently, the interaction in HMD is mostly adopted to ultimately replicate interaction behavior in the original website, such as clicking, thumbing up, etc. HMD itself also comes with more comprehensive capabilities to capture and understand humans, which might inspire new designs for a better collaboration experience across PC and HMD.

⁵<https://support.apple.com/zh-cn/guide/mac-help/mchlbf5b039/mac>

⁶<https://openai.com/blog/chatgpt>

- **Benchmarking User Experience.** The benefits of the system, although intuitive, have not been quantitatively evaluated. We plan on evaluating the improvements in the interaction experience in a qualitative manner in our future work.

We believe WebJump shows its potential to make the best use of current commercial HMDs and existing high-quality screens. We will continue to improve WebJump in the future.

ACKNOWLEDGMENTS

We thank Benfeng Xu for helping to revise our first draft. The work is supported by National Natural Science Foundation of China under Grant No. 6210240, and Academy of Intelligent Computing Technology, Shandong Institutes of Industrial Technology under Grant No.SDAICT2291020, and Beijing Municipal Science & Technology Commission (No. Z221100002722009), and technology innovation Program of Hunan Province (No.2021GK4015).

REFERENCES

- [1] Frederik Brudy, David Ledo, Michel Pahud, Nathalie Henry Riche, Christian Holz, Anand Waghumare, Hemant Bhaskar Surale, Marcus Peinado, Xiaokuan Zhang, Shannon Joyner, et al. 2020. SurfaceFleet: Exploring distributed interactions unbounded from device, application, user, and time. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology*. 7–21.
- [2] A. Butz, T. Hollerer, S. Feiner, B. MacIntyre, and C. Beshers. 1999. Enveloping users and computers in a collaborative 3D augmented reality. In *Proceedings 2nd IEEE and ACM International Workshop on Augmented Reality (IWAR'99)*. 35–44. <https://doi.org/10.1109/IWAR.1999.803804>
- [3] Pei-Yu (Peggy) Chi and Yang Li. 2015. Weave: Scripting Cross-Device Wearable Interaction. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems* (Seoul, Republic of Korea) (CHI '15). Association for Computing Machinery, New York, NY, USA, 3923–3932. <https://doi.org/10.1145/2702123.2702451>
- [4] Tom Horak, Andreas Mathisen, Clemens N. Klokmoose, Raimund Dachselt, and Niklas Elmqvist. 2019. Vistribute: Distributing Interactive Visualizations in Dynamic Multi-Device Setups. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland UK) (CHI '19). Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3290605.3300846>
- [5] Zhanpeng Huang, Weikai Li, and Pan Hui. 2015. Ubii: Towards Seamless Interaction between Digital and Physical Worlds. In *Proceedings of the 23rd ACM international conference on Multimedia - MM '15*. ACM Press, Brisbane, Australia, 341–350. <https://doi.org/10.1145/2733373.2806266>
- [6] Zhen Li, Michelle Annett, Ken Hinckley, Karan Singh, and Daniel Wigdor. 2019. HoloDoc: Enabling Mixed Reality Workspaces that Harness Physical and Digital Content. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems (CHI '19)*. Association for Computing Machinery, New York, NY, USA, 1–14. <https://doi.org/10.1145/3290605.3300917>
- [7] Michael Nebeling. 2017. Xdbrowser 2.0: Semi-automatic generation of cross-device interfaces. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 4574–4584.
- [8] Michael Nebeling and Anind K Dey. 2016. XDBrowser: user-defined cross-device web page designs. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. 5494–5505.
- [9] Erwan Normand and Michael J. McGuffin. 2018. Enlarging a Smartphone with AR to Create a Handheld VESAD (Virtually Extended Screen-Aligned Display). In *2018 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. 123–133. <https://doi.org/10.1109/ISMAR.2018.00043>
- [10] Patrick Reipschläger and Raimund Dachselt. 2019. DesignAR: Immersive 3D-Modeling Combining Augmented Reality with Interactive Displays. In *Proceedings of the 2019 ACM International Conference on Interactive Surfaces and Spaces (Daejeon, Republic of Korea) (ISS '19)*. Association for Computing Machinery, New York, NY, USA, 29–41. <https://doi.org/10.1145/3343055.3359718>
- [11] Pejman Saeghe, Gavin Abercrombie, Bruce Weir, Sarah Clinch, Stephen Pettifer, and Robert Stevens. 2020. Augmented Reality and Television: Dimensions and Themes. In *ACM International Conference on Interactive Media Experiences* (Cornella, Barcelona, Spain) (IMX '20). Association for Computing Machinery, New York, NY, USA, 13–23. <https://doi.org/10.1145/3391614.3393649>
- [12] Steven Schirra and Frank R. Bentley. 2015. "It's Kind of like an Extra Screen for My Phone": Understanding Everyday Uses of Consumer Smart Watches. In *Proceedings of the 33rd Annual ACM Conference Extended Abstracts on Human Factors in Computing Systems* (Seoul, Republic of Korea) (CHI EA '15). Association for Computing Machinery, New York, NY, USA, 2151–2156. <https://doi.org/10.1145/2702613.2732931>
- [13] Marcos Serrano, Barrett Ens, Xing-Dong Yang, and Pourang Irani. 2015. Gluey: Developing a Head-Worn Display Interface to Unify the Interaction Experience in Distributed Display Environments. In *Proceedings of the 17th International Conference on Human-Computer Interaction with Mobile Devices and Services - MobileHCI '15*. ACM Press, Copenhagen, Denmark, 161–171. <https://doi.org/10.1145/2785830.2785838>
- [14] Dirk Wenig, Johannes Schöning, Alex Olwal, Mathias Oben, and Rainer Malaka. 2017. WatchThru: Expanding Smartwatch Displays with Mid-Air Visuals and Wrist-Worn Augmented Reality. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems* (Denver, Colorado, USA) (CHI '17). Association for Computing Machinery, New York, NY, USA, 716–721. <https://doi.org/10.1145/3025453.3025852>
- [15] Jishuo Yang and Daniel Wigdor. 2014. Panelrama: Enabling Easy Specification of Cross-Device Web Applications. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Toronto, Ontario, Canada) (CHI '14). Association for Computing Machinery, New York, NY, USA, 2783–2792. <https://doi.org/10.1145/2556288.2557199>
- [16] Fengyuan Zhu and Tovi Grossman. 2020. BISHARE: Exploring Bidirectional Interactions Between Smartphones and Head-Mounted Augmented Reality. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI '20). Association for Computing Machinery, New York, NY, USA, 1–14. <https://doi.org/10.1145/3313831.3376233>